

DOCUMENTACIÓN TÉCNICA · SEGURIDAD

Spring Security de punta a punta Tienda de Juegos — arquitectura de microservicios

Recorrido ordenado por archivo y por método: qué hace cada clase de seguridad, cómo se encadenan los filtros, cómo viaja la identidad dentro del JWT y dónde se decide cada permiso.

Proyecto: tienda-juegos-spring-security · **Módulos cubiertos:** common-security, api-gateway, auth-service, usuarios-service, catalogo-service

Stack: Spring Boot 3 · Spring Security 6 · Spring Cloud Gateway (WebFlux) · JWT 0.12.6 · RSA / RS256

Fecha: 7 de septiembre de 2026

Contenido

1. 1 Panorama: cómo está partida la seguridad
2. 2 Los dos flujos clave (login y request protegida)
3. 3 Módulo `common-security` `librería`
4. 4 `api-gateway` `WebFlux`
5. 5 `auth-service` `servlet` — el único que firma
6. 6 `usuarios-service` `servlet`
7. 7 `catalogo-service` `servlet`
8. 8 Claves RSA (`keys/`)
9. 9 Tests que fijan el comportamiento de seguridad
10. 10 Tablas de referencia y notas de producción

1 Panorama: cómo está partida la seguridad

El sistema era un monolito Spring MVC con sesión HTTP y un secreto HMAC compartido. Al partirlo en microservicios, la seguridad se reorganizó sobre tres ideas:

1. **Un solo emisor de identidad.** Sólo `auth-service` tiene la clave privada RSA y por lo tanto es el único que puede *firmar* access tokens. El resto sólo tiene la clave pública: *validan*, no emiten.
2. **Identidad autocontenida en el JWT.** El token lleva `sub` (username), `uid` (id numérico), `rol` e `iss`. Ningún servicio consulta una tabla de usuarios para autenticar: la identidad "es lo que el token afirma".
3. **Defensa en profundidad.** El gateway valida el token de "primera línea" y rechaza rápido lo que no lo trae; **igual** cada servicio lo vuelve a validar por su cuenta con su propio `SecurityFilterChain`.

Piezas y responsabilidades

Módulo	Tipo	Rol en seguridad	Claves
<code>common-security</code>	librería	JWT (generar/validar), filtro servlet que puebla el <code>SecurityContext</code> , filtro de token interno, autoconfiguración.	—
<code>api-gateway</code>	WebFlux	Punto de entrada único. Limpia headers sensibles, valida el JWT antes de rutear, propaga la identidad como <code>X-Auth-*</code> , maneja CORS, sirve el React.	pública
<code>auth-service</code>	servlet	Login usuario/contraseña, bloqueo por intentos, 2FA (TOTP), OAuth2 con Google, emisión y rotación de tokens.	pública + privada
<code>usuarios-service</code>	servlet	100% stateless. Todo <code>/api/usuarios/**</code> y <code>/api/admin/**</code> es sólo ADMIN.	pública
<code>catalogo-service</code>	servlet	Reglas por rol para publicar/comprar + autorización "a nivel de dato" (¿es tu juego?).	pública

POR QUÉ RSA Y NO HMAC

Con un secreto HMAC compartido, cualquier servicio que valide también podría falsificar tokens. Con un par asimétrico, el poder de *emitir* queda aislado en `auth-service`; los demás reciben una clave que sólo sirve para verificar la firma.

Roles del dominio

Definidos en `usuarios-service` como el enum `Rol { ADMIN, VENDEDOR, COMPRADOR }`. Dentro del token viajan como el string del claim `rol`; Spring los ve como la autoridad `ROLE_ADMIN / ROLE_VENDEDOR / ROLE_COMPRADOR` (el prefijo `ROLE_` lo agrega `AuthPrincipal.authorities()`). Hay además un rol sintético `ROLE_INTERNAL` que sólo existe para las llamadas servicio-a-servicio.

2 Los dos flujos clave

A · Login usuario / contraseña

```
React —POST /api/auth/login—> api-gateway
      | (ruta /api/auth/** => NO exige token)
      ▼
      auth-service : AuthController.login()
      ▼
      AuthService.login()
1. authenticationManager.authenticate(user, pass)
   └─ DaoAuthenticationProvider
      └─ CredentialUserDetailsService.loadUserByUsername() (lee tabla credenciales)
      └─ BCrypt.matches(pass, hash)
      └─ isAccountNonLocked() (bloqueo por intentos)
      └─ publica AuthenticationSuccess/FailureEvent —> LoginAttemptListener
2. usuariosClient.porUsername() —X-Internal-Token—> usuarios-service (rol + habilitado)
3. si credential.totpHabilitado => TotpService.validar(secret, codigo)
4. jwtService.generar(uid, username, rol) => access token (RS256, 15 min)
5. refreshTokenService.crear(username) => refresh token (UUID, 7 días, en BD)
      ▼
      LoginResponse { token, refreshToken, usuario }
```

B · Request protegida (p. ej. **DELETE /api/juegos/42**)

```
React —DELETE /api/juegos/42 (Authorization: Bearer eyJ...)—> api-gateway
GatewayAuthFilter (order -1)
  · borra X-Internal-Token, X-Auth-* entrantes (anti-spoofing)
  · path == /api/** y != /api/auth/** => protegido
  · JwtValidator.validar(token) -> firma RSA + issuer OK
  · agrega X-Auth-Username / X-Auth-Uid / X-Auth-Rol
      ▼ (ruteo Spring Cloud Gateway)
      catalogo-service
SecurityFilterChain:
  · InternalTokenFilter -> shouldNotFilter (no es /internal/**) -> pasa
  · JwtAuthenticationFilter -> valida token, arma AuthPrincipal, setea SecurityContext
  · authorizeHttpRequests -> DELETE /api/juegos/** requiere hasAnyRole(VENDEDOR, ADMIN)
      ▼
JuegoController.eliminar()
  · @PreAuthorize("hasAnyRole('VENDEDOR','ADMIN')") (chequeo de rol, otra vez)
  · verificarPropietarioOAdmin(juego, principal)
    └─ JuegoService.puedeGestionar() -> principal.isAdmin() || juego.vendedorUsername == prin
  · si no => AccessDeniedException => 403 JSON
```

LECTURA CLAVE

El rol se chequea hasta

TRES

veces (gateway, `authorizeHttpRequests`, `@PreAuthorize`) y la *pertenencia* del dato una vez, en el servicio. Es redundante a propósito: cada capa no confía en que la anterior haya corrido.

3 Módulo common-security librería

JAR puro (no ejecutable). Con sólo tenerlo en el classpath, un servicio servlet recibe `JwtService` y `JwtAuthenticationFilter` listos. Orden de lectura de abajo hacia arriba en la cadena de dependencias: propiedades → claves → servicio JWT → principal → filtros → autoconfiguración.

JwtProperties

`common-security/.../security/JwtProperties.java`

`@ConfigurationProperties(prefix = "app.jwt")`. Es el binding de las claves `app.jwt.*` del `application.properties` de cada servicio. No tiene lógica: sólo campos + getters/setters.

Propiedad	Default	Para qué
<code>public-key-location</code>	—	Siempre presente. Clave pública RSA (PEM X.509) para validar la firma de tokens entrantes.
<code>private-key-location</code>	<code>null</code>	Sólo <code>auth-service</code> . Clave privada (PKCS#8) para firmar. Si falta, <code>generar()</code> lanza excepción.
<code>access-expiration-ms</code>	<code>900000</code> (15 min)	Vida del access token.
<code>issuer</code>	<code>gamestore-auth</code>	Claim <code>iss</code> ; los validadores lo exigen.

PemKeys (package-private, utilitaria)

`common-security/.../security/PemKeys.java`

Convierte PEM (texto Base64 de `openssl`) en objetos `java.security.Key`. Sin BouncyCastle: alcanza el JDK.

```
static RSAPublicKey readPublicKey(Resource pem)
```

- 1 Llama a `der(readPem(pem), "PUBLIC KEY")`: lee el archivo como texto y saca los encabezados `-----BEGIN/END PUBLIC KEY-----` y todos los blancos.
- 2 Base64-decodifica lo que queda → bytes DER.
- 3 `KeyFactory.getInstance("RSA").generatePublic(new X509EncodedKeySpec(der))` → castea a `RSAPublicKey`.
- 4 Cualquier error se envuelve en `IllegalStateException` nombrando el archivo (falla temprano y clara).

```
static RSAPrivateKey readPrivateKey(Resource pem)
```

- 1 Igual que la anterior pero con marca `"PRIVATE KEY"` y `PKCS8EncodedKeySpec`.
- 2 Si el PEM no es PKCS#8 (p. ej. el viejo `-----BEGIN RSA PRIVATE KEY-----`), el mensaje de error lo dice explícitamente.

```
private static String readPem(Resource) · byte[] der(String, String) · String  
describe(Resource)
```

- 1 `readPem`: `StreamUtils.copyToString` del `InputStream` del recurso, UTF-8.
- 2 `der`: `replace` de los delimitadores + `replaceAll("\\s", "")` + `Base64.getDecoder().decode`.
- 3 `describe`: devuelve el URI del recurso para los mensajes de error (o la descripción si no hay URI).

JwtService

common-security/.../security/JwtService.java

Genera y valida access tokens RS256. Se instancia como bean en la autoconfiguración.

```
JwtService(JwtProperties props) · constructor
```

- 1 Si `publicKeyLocation == null` → `IllegalStateException` ("Falta app.jwt.public-key-location").
- 2 `this.publicKey = PemKeys.readPublicKey(...)`.
- 3 Si hay `privateKeyLocation` → `this.privateKey = PemKeys.readPrivateKey(...)`; si no, `privateKey = null`. **Ese null es lo que separa a auth-service del resto.**
- 4 Guarda `expiracionMs` e `issuer`.

```
String generar(long uid, String username, String rol)
```

- 1 Si `privateKey == null` → `IllegalStateException` ("solo auth-service firma"). Fail-fast: un servicio que no debería emitir no lo hace por accidente.
- 2 Construye con `Jwts.builder(): issuer(issuer), subject(username), claim("uid", uid), claim("rol", rol), issuedAt(ahora), expiration(ahora + expiracionMs)`.
- 3 `.signWith(privateKey, Jwts.SIG.RS256).compact()` → string del JWT.

```
Jws<Claims> validar(String token)
```

- 1 `Jwts.parser().verifyWith(publicKey).requireIssuer(issuer).build().parseSignedClaims(token)`.
- 2 Lanza `JwtException` si el token está corrupto, mal firmado, expirado o con otro `iss`. No devuelve `null`: o parsea o explota.

```
AuthPrincipal aPrincipal(Claims claims)
```

- 1 Lee `uid` como `Number` (tolera int/long) → `longValue()`, o `0L` si falta.
- 2 Devuelve `new AuthPrincipal(uid, claims.getSubject(), claims.get("rol", String.class))`.

AuthPrincipal (record)

common-security/.../security/AuthPrincipal.java

La identidad que viaja en el token y queda como `principal` del `SecurityContext` en cada servicio. Reemplaza al `CustomUserDetails` del monolito: **no hay entidad JPA detrás**.

```
public record AuthPrincipal(long id, String username, String rol)
```

```
List<GrantedAuthority> authorities()
```

- 1 Devuelve `List.of(new SimpleGrantedAuthority("ROLE_" + rol))`. Acá se agrega el prefijo `ROLE_` que después usan `hasRole(...)` y `@PreAuthorize`.

```
boolean isAdmin()
```

- 1 `return "ADMIN".equals(rol)`. Atajo que usa `catalogo-service` para la autorización a nivel de dato.

JwtAuthenticationFilter servlet

`common-security/.../security/JwtAuthenticationFilter.java`

`extends OncePerRequestFilter`. Pueba el `SecurityContext` a partir del header `Authorization`, **sin tocar la base**. Lo usan `auth-`, `usuarios-` y `catalogo-service` (el gateway hace su versión reactiva).

```
void doFilterInternal(request, response, chain)
```

- 1 Lee `header = request.getHeader("Authorization")`.
- 2 Sigue sólo si `header != null`, empieza con `"Bearer "` y el contexto todavía no tiene autenticación (no pisa la que puso `InternalTokenFilter`).
- 3 `jws = jwtService.validar(header.substring(7))` y `principal = jwtService.aPrincipal(jws.getPayload())`.
- 4 Crea `UsernamePasswordAuthenticationToken(principal, null, principal.authorities())`, le agrega `WebAuthenticationDetails` (IP, session id) y lo mete en `SecurityContextHolder`.
- 5 Si `validar()` tira `JwtException` / `IllegalArgumentException`: se traga la excepción y la request sigue **anónima** — son las reglas del `SecurityFilterChain` las que después deciden 401/403.
- 6 Siempre `chain.doFilter(request, response)`.

InternalTokenFilter servlet

`common-security/.../security/InternalTokenFilter.java`

`extends OncePerRequestFilter`. Protege los endpoints `/internal/**` que un servicio expone **sólo** para que lo llamen otros servicios. Exige el header `X-Internal-Token` con un secreto compartido. Cada servicio lo instancia a mano en su `SecurityConfig` (no es un bean autoconfigurado).

`InternalTokenFilter(String secretoEsperado) · constructor`

- 1 Guarda el secreto esperado (viene de `app.internal.secret`). Constante pública `HEADER = "X-Internal-Token"` reutilizada por los clientes HTTP.

`boolean shouldNotFilter(HttpServletRequest request)`

- 1 `return !request.getRequestURI().startsWith("/internal/");` El filtro sólo corre para rutas internas; todo lo demás lo salta.

`void doFilterInternal(request, response, chain)`

- 1 `recibido = request.getHeader("X-Internal-Token");`
- 2 Si el secreto esperado es `null` /vacío o no coincide con el recibido: responde `401` con `{"mensaje":"Llamada interna no autorizada"}` y **corta** (no llama a `chain`).
- 3 Si coincide: crea un `UsernamePasswordAuthenticationToken("internal-service", null, ["ROLE_INTERNAL"])` y lo pone en el contexto → así las reglas `.hasRole("INTERNAL")` lo dejan pasar.
- 4 `chain.doFilter(request, response);`

NOTA DE DISEÑO

Es deliberadamente simple. En un clúster real se reforzaría con network policies / mTLS y este header sería una segunda barrera. Para la demo (docker-compose en red interna) alcanza. El gateway, además,

BORRA

este header si viene de afuera.

CommonSecurityAutoConfiguration

`common-security/.../security/CommonSecurityAutoConfiguration.java`

`@AutoConfiguration + @EnableConfigurationProperties(JwtProperties.class)`. Registrada en `META-INF/spring/...AutoConfiguration.imports`.

`@Bean JwtService jwtService(JwtProperties props)`

- 1 `@ConditionalOnMissingBean`: si el servicio no definió el suyo, crea `new JwtService(props)`.

`@Bean JwtAuthenticationFilter jwtAuthenticationFilter(JwtService jwtService)`

- 1 `@ConditionalOnMissingBean + @ConditionalOnClass("...OncePerRequestFilter")`: sólo en apps servlet (en el gateway reactivo no existe esa clase, así que no se crea).
- 2 Devuelve `new JwtAuthenticationFilter(jwtService)`. Cada servicio después lo *engancha* en su propia cadena con `addFilterBefore(...)`.

Lo único que el navegador ve. **No** depende de `common-security` a propósito: esa librería arrastra `spring-security-web` (servlet) y confundiría a Boot sobre si la app es servlet o reactiva. El gateway re-implementa la validación mínima con JWT + la clave pública.

ApiGatewayApplication

api-gateway/.../gateway/ApiGatewayApplication.java

`@SpringBootApplication` + `main()` con `SpringApplication.run(...)`. Sin lógica; el Javadoc resume las 4 tareas del gateway: rutear, servir React, validar JWT de primera línea, borrar headers sensibles.

JwtValidator

api-gateway/.../gateway/JwtValidator.java

`@Component`. Misma clave pública y mismo issuer que el resto, pero código propio (no comparte `JwtService`).

```
JwtValidator(@Value("${app.jwt.public-key-location}") Resource,
@Value("${app.jwt.issuer:gamestore-auth}") String) · constructor
```

- 1 Guarda el `issuer`.
- 2 `this.publicKey = leerClavePublica(pem)`.

```
Optional<Claims> validar(String token)
```

- 1 `Jwts.parser().verifyWith(publicKey).requireIssuer(issuer).build().parseSignedClaims(token)`.
- 2 Si parsea → `Optional.of(jws.getPayload())`.
- 3 Si tira cualquier `RuntimeException` (firma mala, expirado, issuer distinto) → `Optional.empty()`. El gateway nunca deja escapar una excepción de parseo.

```
private static RSAPublicKey leerClavePublica(Resource pem)
```

- 1 Lee el PEM a string, saca delimitadores `PUBLIC KEY` y blancos, Base64-decode → DER.
- 2 `KeyFactory RSA` + `X509EncodedKeySpec` → `RSAPublicKey`. Error → `IllegalStateException` al arranque.
- 3 (Es una copia reducida de `PemKeys`, justificada por no querer la dependencia servlet.)

GatewayAuthFilter — el filtro global

api-gateway/.../gateway/GatewayAuthFilter.java

`@Component implements GlobalFilter, Ordered`. Corre en **todo** request que pasa por el gateway.

```
Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
```

- 1 **Limpieza anti-spoofing.** `mutado = exchange.getRequest().mutate()` y por cada header de `["X-Internal-Token", "X-Auth-Username", "X-Auth-Uid", "X-Auth-Rol"]` hace `headers.remove(h)`. Un cliente no puede hacerse pasar por otro servicio ni inyectar identidad.
- 2 **¿Ruta protegida?** `protegido = path.startsWith("/api/") && !path.startsWith("/api/auth/").` Login/refresh/logout quedan públicos.
- 3 Si es protegida: toma `Authorization`. Si falta o no empieza con `"Bearer "` → `rechazar(exchange, "Falta el token de autenticacion")`.
- 4 `claims = jwtValidator.validar(token)`. Si `isEmpty()` → `rechazar(exchange, "Token invalido o expirado")`.
- 5 Si el token es válido: agrega hacia adentro `X-Auth-Username` (= `sub`), `X-Auth-Uid` (si hay `uid`) y `X-Auth-Rol`. Es cortesía para un servicio futuro que no quiera re-parsear.
- 6 `return chain.filter(exchange.mutate().request(mutado.build()).build())` — sigue con la request mutada.

```
private Mono<Void> rechazar(ServerWebExchange exchange, String mensaje)
```

- 1 Setea `401`, `Content-Type: application/json`.
- 2 Escribe el body `{"mensaje":"..."}` con `response.writeWith(Mono.just(...wrap(bytes)))`. No toca a los servicios de atrás.

```
int getOrder()
```

- 1 `return -1` → corre **antes** del filtro de ruteo de Spring Cloud Gateway.

SpaFallbackConfig

api-gateway/.../gateway/SpaFallbackConfig.java

@Configuration. No es seguridad estrictamente, pero define qué es público servir.

```
@Bean RouterFunction<ServerResponse> spaRouter()
```

- 1 Ruta `GET("/")` y **no** `/api/**`.
- 2 Si existe `classpath:static/index.html` (build de React) → lo devuelve como `text/html`.
- 3 Si no está compilado → devuelve un texto guía en `text/plain` (evita un 500).

application.yml — CORS y rutas

api-gateway/src/main/resources/application.yml

- **CORS** (`spring.cloud.gateway.globalcors`): `allowed-origin-patterns` = `${GATEWAY_ALLOWED_ORIGINS:*` — en dev cualquier origen; en prod se obliga a pasar la URL de Vercel.
`allowed-methods`: GET/POST/PUT/DELETE/OPTIONS. `allowed-headers`: Authorization, Content-Type.

`allow-credentials: false` (la API usa el token en header, no cookies, así que aceptar cualquier origen no expone nada).

- **Rutas:** `/api/auth/**` → auth-service; `/api/usuarios/**, /api/admin/**` → usuarios-service; `/api/juegos/**, /api/secciones/**, /api/compras/**` y `/uploads/**` → catalogo-service.
- **JWT:** `app.jwt.public-key-location` (sólo pública) y `app.jwt.issuer`.

CORS VIVE SÓLO ACÁ

Los servicios de atrás

NO

configuran CORS. Sólo el gateway habla con navegadores. Si `auth-service` validara `Origin`, rechazaría el header que el gateway le reenvía y respondería 403.

5 `auth-service` `servlet` — el único que firma

Depende de `common-security`, `spring-boot-starter-security` y `spring-boot-starter-oauth2-client`. Tiene la clave privada.

`SecurityConfig`

`auth-service/.../security/SecurityConfig.java`

`@Configuration @EnableWebSecurity`. Dos "puertas": `/api/**` (stateless salvo el flujo OAuth2) e `/internal/**` (otros servicios).

```
@Bean PasswordEncoder passwordEncoder()
```

- 1 `return new BCryptPasswordEncoder()`. Todos los hashes de `credenciales` son BCrypt.

```
@Bean DaoAuthenticationProvider authenticationProvider(UserDetailsService uds,  
PasswordEncoder encoder)
```

- 1 Crea el provider, le setea el `UserDetailsService` (= `CredentialUserDetailsService`) y el `PasswordEncoder`.
- 2 Este provider es el que compara el hash y consulta `isAccountNonLocked()`.

```
@Bean AuthenticationManager authenticationManager(DaoAuthenticationProvider provider,  
ApplicationEventPublisher eventPublisher)
```

- 1 Envuelve el provider en un `ProviderManager` (no un lambda suelto) **para** que se publiquen los `AuthenticationSuccessEvent` / `...FailureEvent`.
- 2 `manager.setAuthenticationEventPublisher(new DefaultAuthenticationEventPublisher(eventPublisher))`. De esos eventos vive `LoginAttemptListener`.

```
@Bean SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationFilter
jwtAuthenticationFilter, OAuth2LoginSuccessHandler oAuth2LoginSuccessHandler)
```

- 1 `cors().disable()`, `csrf().disable()`, `formLogin().disable()`, `httpBasic().disable()`.
- 2 `sessionManagement(IF_REQUIRED)` — **no** STATELESS: el login con Google necesita sesión HTTP para su baile de redirects.
- 3 **Reglas** `authorizeHttpRequests`, en orden: `/actuator/health*` → `permitAll`; `/internal/**` → `hasRole("INTERNAL")`; `/api/auth/login|refresh|logout|me` → `permitAll`; `/oauth2/**`, `/login/oauth2/**` → `permitAll`; `/api/auth/2fa/**` → `authenticated`; `anyRequest()` → `authenticated`.
- 4 `oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler))`.
- 5 `exceptionHandling`: `authenticationEntryPoint` → 401 JSON `{"mensaje":"No autenticado"}`; `accessDeniedHandler` → 403 JSON `{"mensaje":"No tenes permiso..."}`.
- 6 **Filtros añadidos** antes de `UsernamePasswordAuthenticationFilter`: primero `new InternalTokenFilter(internalSecret)`, después `jwtAuthenticationFilter`. Orden importa: si entró por token interno, el JWT filter ve el contexto ya lleno y no lo pisa.
- 7 `return http.build()`.

CredentialUserDetails implements UserDetails

auth-service/.../security/CredentialUserDetails.java

Adaptador `Credential` → `UserDetails`, sólo para que el `DaoAuthenticationProvider` compare el hash y aplique el bloqueo.

Método	Devuelve
<code>getPassword()</code>	<code>credential.getPasswordHash()</code> (el BCrypt a comparar).
<code>getUsername()</code>	<code>credential.getUsername()</code> .
<code>getAuthorities()</code>	<code>List.of()</code> — el rol no vive acá, lo trae usuarios-service.
<code>isAccountNonLocked()</code>	<code>bloqueadoHasta == null bloqueadoHasta.isBefore(now())</code> — acá se hace efectivo el bloqueo por intentos : si está en el futuro, el provider lanza <code>LockedException</code> .
<code>isAccountNonExpired()</code> / <code>isCredentialsNonExpired()</code>	<code>true</code> fijo.
<code>isEnabled()</code>	<code>true</code> siempre — el flag "habilitado" lo chequea <code>AuthService</code> después, contra usuarios-service.
<code>getCredential()</code>	La entidad, para que <code>AuthService</code> y <code>LoginAttemptListener</code> la usen tras el login.

CredentialUserDetailsService implements UserDetailsService

auth-service/.../security/CredentialUserDetailsService.java

```
UserDetails loadUserByUsername(String username)
```

- 1 `credentialRepository.findByUsername(username)`.
- 2 `.map(CredentialUserDetails::new)`.
- 3 `.orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado: " + username))`
→ el provider lo traduce a `BadCredentialsException`.

LoginAttemptListener

auth-service/.../security/LoginAttemptListener.java

`@Component`. Bloqueo automático: `MAX_INTENTOS = 5` → `BLOQUEO_MINUTOS = 15`. Funciona por eventos del `AuthenticationManager`; el estado se guarda en `Credential`.

```
@EventListener void onFailure(AbstractAuthenticationFailureEvent event)
```

- 1 `username = event.getAuthentication().getName()`.
- 2 `credentialRepository.findByUsername(username).ifPresent(cred -> { ... })`.
- 3 Si ya está bloqueado y el bloqueo sigue vigente → `return` (no sigue sumando).
- 4 `cred.setIntentosFallidos(cred.getIntentosFallidos() + 1)`.
- 5 Si `intentosFallidos >= 5` → `setBloqueadoHasta(now().plusMinutes(15))` y resetea el contador a 0.
- 6 `credentialRepository.save(cred)`.

```
@EventListener void onSuccess(AuthenticationSuccessEvent event)
```

- 1 Si el principal **no** es `CredentialUserDetails` → `return` (el login con Google usa otro principal).
- 2 Si `cred` tiene intentos > 0 o `bloqueadoHasta` seteado → los limpia (`0` / `null`) y guarda. Un login exitoso "perdona" los intentos previos.

TotpService

auth-service/.../security/TotpService.java

`@Service`. TOTP (RFC 6238) a mano — el mismo algoritmo de Google Authenticator. `DIGITOS = 6`, `PASO_SEGUNDOS = 30`, `VENTANA_TOLERANCIA = 1`.

```
String generarSecreto()
```

- 1 20 bytes de `SecureRandom` → `Base32.encodeToString(...)`. Es lo que se muestra al usuario / se mete en el QR.

```
String generarUri(String secreto, String usernameCuenta)
```

- 1 Arma la URI `otpauth://totp/GameStore:usuario?secret=...`
`&issuer=GameStore&algorithm=SHA1&digits=6&period=30` (URL-encoding de issuer y cuenta). El frontend la convierte en QR.

```
boolean validar(String secreto, String codigo)
```

- 1 Si `codigo` es null o no matchea `\d{6}` → `false`.
- 2 `ventanaActual = epochSegundos / 30`.
- 3 Prueba `i ∈ [-1, +1]`: si `codigo.equals(generarCodigo(secreto, ventanaActual + i))` para alguna → `true` (tolera relojes desfasados 30 s).
- 4 Si ninguna coincide → `false`.

```
private String generarCodigo(String secretoBase32, long contador)
```

- 1 `Base32.decode` del secreto → clave HMAC.
- 2 `contador` → 8 bytes big-endian.
- 3 `HmacSHA1(clave, bytesContador)` → hash de 20 bytes.
- 4 **Truncado dinámico**: `offset = hash[19] & 0x0f`; toma 4 bytes desde `offset`, limpia el bit de signo, arma un int de 31 bits.
- 5 `int % 10^6` → `String.format("%06d", ...)`. Error interno → `IllegalStateException`.

OAuth2ClientConfig

auth-service/.../security/OAuth2ClientConfig.java

`@Configuration`. Registra el cliente de Google a mano para que el login con Google sea **opcional de verdad**.

```
@Bean ClientRegistrationRepository clientRegistrationRepository()
```

- 1 Si `app.oauth2.google.client-id` o `client-secret` están en blanco → devuelve `registrationId - > null` (repo vacío; la app arranca igual, sin OAuth2).
- 2 Si están:
`CommonOAuth2Provider.GOOGLE.getBuilder("google").clientId(...).clientSecret(...).build()`.
- 3 `return new InMemoryClientRegistrationRepository(google)`.

OAuth2LoginSuccessHandler implements AuthenticationSuccessHandler

auth-service/.../security/OAuth2LoginSuccessHandler.java

`@Component`. Corre cuando Google confirma la identidad. Convierte esa sesión OAuth2 en **nuestro** par de tokens.

```
void onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response,
Authentication authentication)
```

- 1 `oauthUser = (OAuth2User) authentication.getPrincipal();` lee `email` y `name` de los atributos.
- 2 `info = usuariosClient.buscarOCrearGoogle(email, nombre)` — usuarios-service decide el rol (COMPRADOR por defecto) y devuelve id/username/rol.
- 3 `credentialService.asegurarParaGoogle(info.username(), email)` — crea una credencial local con password random inutilizable si no existe (para que después pueda activar 2FA y para que el refresh token tenga a quién apuntar).
- 4 `accessToken = jwtService.generar(info.id(), info.username(), info.rol());`
- 5 `refreshToken = refreshTokenService.crear(info.username()).getToken();`
- 6 `response.sendRedirect(frontendUrl + "/#/oauth-callback?token=...&refreshToken=...")` (ambos URL-encoded). El SPA levanta los tokens del fragment.

Modelos: **Credential** y **RefreshToken**

auth-service/.../model/Credential.java · RefreshToken.java

Credential (`@Entity`, tabla `credenciales`): la parte "secreta" del usuario, separada del perfil. Campos: `id`, `username` (único, clave natural compartida), `passwordHash` (BCrypt), `email` (copia, para casar la cuenta en el login con Google), `intentosFallidos`, `bloqueadoHasta`, `totpSecret`, `totpHabilitado`. **No** tiene rol ni flag "habilitado" a propósito.

RefreshToken (`@Entity`, tabla `refresh_tokens`): string opaco (UUID) de larga vida, guardado en BD **para poder revocarlo** (un JWT suelto no se invalida antes de expirar). Campos: `token` (único), `username`, `expiraEn`, `revocado`, `creadoEn`.

```
boolean RefreshToken.esValido()
```

- 1 `return !revocado && expiraEn.isAfter(LocalDateTime.now());`

AuthService

auth-service/.../service/AuthService.java

`@Service`. Orquesta login / refresh / logout.

```
LoginResponse login(String username, String password, String totpCode)
```

- 1 credential = autenticarPassword(username, password) (ver abajo).
- 2 info = usuariosClient.porUsername(username). Si !info.habilitado() → AuthExceptions.CuentaDeshabilitada.
- 3 Si credential.isTotpHabilitado(): si falta totpCode → DosFactoresRequerido("Ingresa el codigo..."); si !totpService.validar(secret, totpCode) → DosFactoresRequerido("Codigo... incorrecto").
- 4 accessToken = jwtService.generar(info.id(), username, info.rol()).
- 5 refresh = refreshTokenService.crear(username).
- 6 return new LoginResponse(accessToken, refresh.getToken(), aDto(info)).

```
private Credential autenticarPassword(String username, String password)
```

- 1 authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(username, password)).
- 2 Éxito → ((CredentialUserDetails) result.getPrincipal()).getCredential().
- 3 catch LockedException → NoAutorizado("Cuenta bloqueada temporalmente..").
- 4 catch BadCredentialsException / AuthenticationException → NoAutorizado("Usuario o contraseña incorrectos") (mismo mensaje: no filtra si el usuario existe).

```
RefreshResponse refresh(String refreshToken)
```

- 1 actual = refreshTokenService.buscarValido(refreshToken).orElseThrow(() -> NoAutorizado("Sesion expirada...")).
- 2 info = usuariosClient.porUsername(actual.getUsername()) — re-lee el rol (pudo cambiar).
- 3 nuevo = refreshTokenService.rotar(actual) — revoca el viejo, emite uno nuevo.
- 4 accessToken = jwtService.generar(info.id(), actual.getUsername(), info.rol()).
- 5 return new RefreshResponse(accessToken, nuevo.getToken()).

```
void logout(String refreshToken)
```

- 1 Si no es null:
refreshTokenService.buscarValido(refreshToken).ifPresent(refreshTokenService::revocar).
Idempotente y silencioso.

```
private static UsuarioDTO aDto(UsuarioInfo i)
```

- 1 Mapea UsuarioInfo → UsuarioDTO (id, username, nombreCompleto, email, rol, habilitado).

RefreshTokenService

auth-service/.../service/RefreshTokenService.java

@Service . refresh-expiration-days = 7 por defecto.

```
RefreshToken crear(String username)
```

- 1 repo.save(new RefreshToken(UUID.randomUUID().toString(), username, now().plusDays(7))).

```
Optional<RefreshToken> buscarValido(String token)
```

- 1 repo.findByToken(token).filter(RefreshToken::esValido) — sólo devuelve si no está revocado ni expirado.

```
@Transactional RefreshToken rotar(RefreshToken actual)
```

- 1 actual.setRevocado(true); repo.save(actual) .
- 2 return crear(actual.getUsername()) . **Cada uso rota:** reusar uno revocado no sirve → señal de que se filtró.

```
@Transactional void revocar(RefreshToken token)
```

- 1 token.setRevocado(true); repo.save(token) .

CredentialService

auth-service/.../service/CredentialService.java

@Service . ABM de credenciales; lo dispara usuarios-service por la API interna cuando el ADMIN gestiona usuarios.

```
@Transactional void crear(String username, String email, String rawPassword)
```

- 1 Si repo.existsByUsername(username) → IllegalArgumentException .
- 2 repo.save(new Credential(username, passwordEncoder.encode(rawPassword), email)) .

```
@Transactional void cambiarPassword(String username, String rawPassword)
```

- 1 Busca la credencial (o IllegalArgumentException), setPasswordHash(encode(rawPassword)), guarda.

```
@Transactional void eliminar(String username)
```

- 1 repo.findByUsername(username).ifPresent(repo::delete) .

```
@Transactional void asegurarParaGoogle(String username, String email)
```

- 1 Si no existe la credencial → la crea con `encode(UUID.randomUUID().toString())` (password imposible de adivinar y que nadie conoce).

AuthController

auth-service/.../web/AuthController.java

`@RestController @RequestMapping("/api/auth")`. Todos los endpoints salvo `/me` son `permitAll` a nivel de cadena.

```
POST /login → LoginResponse login(@RequestBody LoginRequest req)
```

- 1 Delega en `authService.login(req.username(), req.password(), req.totpCode())`.

```
POST /refresh → RefreshResponse refresh(@RequestBody RefreshRequest req)
```

- 1 `authService.refresh(req.refreshToken())`.

```
POST /logout → ResponseEntity<Void> logout(@RequestBody(required=false) RefreshRequest req)
```

- 1 `authService.logout(req != null ? req.refreshToken() : null)` → `204 No Content`.

```
GET /me → ResponseEntity<?> me(@AuthenticationPrincipal AuthPrincipal principal)
```

- 1 Si `principal == null` → `401` con `ErrorResponse("No autenticado")`. (Es `permitAll`, así que puede llegar sin token.)
- 2 Intenta `usuariosClient porUsername(principal.username())` → `UsuarioDTO` completo.
- 3 Si `ServicioNoDisponibleException` → devuelve `200` con lo que afirma el token (id, username, rol) en vez de un 503: el usuario puede seguir navegando el catálogo.

TotpController

auth-service/.../web/TotpController.java

`@RestController @RequestMapping("/api/auth/2fa")`. Toda la ruta es `authenticated`. Autoservicio sobre la propia `Credential`.

```
private Credential credencialDe(AuthPrincipal principal)
```

- 1 `credentialRepository.findByUsername(principal.username()).orElseThrow(...)`. Siempre opera sobre **tu** credencial, tomada del token — nunca de un parámetro.

```
GET /estado → Map<String,Boolean> estado(@AuthenticationPrincipal AuthPrincipal)
```

1 {"habilitado": credencialDe(principal).isTotpHabilitado()}.

```
POST /setup → TotpSetupResponse setup(@AuthenticationPrincipal AuthPrincipal)
```

- 1 secreto = totpService.generarSecreto(); lo guarda en la credencial con totpHabilitado = false.
- 2 Devuelve {secret, otpauthUri} para el QR.

```
POST /enable → ResponseEntity<?> enable(@AuthenticationPrincipal AuthPrincipal, @RequestBody TotpCodeRequest req)
```

- 1 Si no hay totpSecret → 400 "Primero inicia el setup...".
- 2 Si !totpService.validar(secret, req.codigo()) → 400 "Codigo incorrecto".
- 3 Si valida → totpHabilitado = true, guarda, 200 "2FA activado...".

```
POST /disable → ResponseEntity<?> disable(@AuthenticationPrincipal AuthPrincipal, @RequestBody TotpCodeRequest req)
```

- 1 Si no está habilitado o el código no valida → 400 "Codigo incorrecto".
- 2 Si OK → totpHabilitado = false, totpSecret = null, guarda, 200 "2FA desactivado".

InternalCredencialesController

auth-service/.../web/InternalCredencialesController.java

@RestController @RequestMapping("/internal/credenciales") . Sólo lo llama usuarios-service (header X-Internal-Token, validado por InternalTokenFilter + regla hasRole("INTERNAL")). Nunca se expone por el gateway.

```
POST / → crear(@RequestBody Map<String,String> body)
```

- 1 credentialService.crear(body.get("username"), body.get("email"), body.get("password")) → 201.

```
PUT /{username}/password → cambiarPassword(@PathVariable String username, @RequestBody Map body)
```

- 1 credentialService.cambiarPassword(username, body.get("password")) → 204.

```
DELETE /{username} → eliminar(@PathVariable String username)
```

1 `credentialService.eliminar(username)` → `204`.

Errores: `AuthExceptions` + `ApiExceptionHandler`

`auth-service/.../service/AuthExceptions.java` · `web/ApiExceptionHandler.java`

Excepción	Handler	Respuesta
<code>DosFactoresRequerido</code>	<code>handle2fa</code>	<code>401</code> {mensaje, requiere2fa:true}
<code>NoAutorizado</code> , <code>CuentaDeshabilitada</code>	<code>handleNoAutorizado</code>	<code>401</code> {mensaje}
<code>ServicioNoDisponibleException</code>	<code>handleDown</code>	<code>503</code> "proba de nuevo en unos segundos"
<code>IllegalArgumentException</code>	<code>handleIllegalArgument</code>	<code>400</code> {mensaje}

`UsuariosClient` (auth-service)

`auth-service/.../client/UsuariosClient.java`

`@Component`. Cliente HTTP hacia la API interna de usuarios-service. Constructor: `RestClient` con `baseUrl` y `defaultHeader(InternalTokenFilter.HEADER, internalSecret)` — **toda** llamada sale con el token interno.

```
@Retry @CircuitBreaker UsuarioInfo porUsername(String username)
```

- 1 GET `/internal/usuarios/by-username/{u}` → `UsuarioInfo`.
- 2 `@Retry`: reintentar ante fallo transitorio. `@CircuitBreaker`: si usuarios-service está caído, tras N fallos va directo al `fallback`.

```
@Retry @CircuitBreaker UsuarioInfo buscarOCrearGoogle(String email, String nombre)
```

- 1 POST `/internal/usuarios/google` con {email, nombre}.

```
private UsuarioInfo fallback(... , Throwable t)
```

- 1 Lanza `ServicioNoDisponibleException("usuarios-service no responde", t)` → el handler lo vuelve 503.

`DataInitializer` (auth-service)

`auth-service/.../config/DataInitializer.java`

```
void run(String... args) · CommandLineRunner
```

- 1 Siembra credenciales de prueba idempotentemente: `admin/admin123`, `vendedor1..2/vendedor123`, `comprador1..5/comprador123`.
- 2 `crearSiFalta`: si `existsByUsername` → nada; si no → `save(new Credential(username, encode(pass), email))`. usuarios-service siembra los perfiles equivalentes por su lado; casan por `username`.

`application.properties` (auth-service) — lo relevante a seguridad

```
# auth-service es el UNICO con clave privada
app.jwt.public-key-location=classpath:jwt-public.pem
app.jwt.private-key-location=classpath:jwt-private.pem
app.jwt.access-expiration-ms=900000 # 15 min
app.jwt.issuer=gamestore-auth
app.jwt.refresh-expiration-days=7
app.internal.secret=dev-internal-secret-cambiar-en-prod
app.oauth2.google.client-id=${GOOGLE_CLIENT_ID}
app.oauth2.google.client-secret=${GOOGLE_CLIENT_SECRET}
```

En el perfil `docker` las mismas claves se pueden montar como secreto

(`APP_JWT_PRIVATE_KEY_LOCATION=file:/run/keys/...`) y `INTERNAL_SECRET` viene por entorno.

100% stateless. Cada request trae un JWT firmado por auth-service o el header `X-Internal-Token`.

SecurityConfig

usuarios-service/.../config/SecurityConfig.java

`@Configuration @EnableWebSecurity @EnableMethodSecurity(prePostEnabled = true)` (habilita `@PreAuthorize`).

```
@Bean SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationFilter
    jwtAuthenticationFilter)
```

- 1 `csrf().disable()`, `formLogin().disable()`, `httpBasic().disable()`.
- 2 `sessionManagement(STATELESS)` — sin sesión, nunca.
- 3 **Reglas:** `/actuator/health*` → `permitAll`; `/internal/**` → `hasRole("INTERNAL")`; `/api/usuarios/**` y `/api/admin/**` → `hasRole("ADMIN")`; `anyRequest()` → `authenticated`.
- 4 `exceptionHandling`: 401 / 403 en JSON, igual que los otros servicios.
- 5 `addFilterBefore(new InternalTokenFilter(internalSecret), UsernamePasswordAuthenticationFilter.class)` y luego `addFilterBefore(jwtAuthenticationFilter, ...)`.

Autorización de método en los controllers

usuarios-service/.../web/UsuarioController.java · AdminStatsController.java

```
@RestController @RequestMapping("/api/usuarios") @PreAuthorize("hasRole('ADMIN')") ·
    UsuarioController
```

- 1 El `@PreAuthorize` a nivel de **clase** aplica a todos los métodos (doble llave con la regla de URL).
- 2 `GET /` → `listar()`: `usuarioService.listarTodos()` → `UsuarioDTO::from`.
- 3 `POST /` → `crear(CrearUsuarioRequest)`: `usuarioService.crearUsuario(username, password, nombreCompleto, email, rol)` — internamente crea el perfil local y delega la credencial en auth-service vía `AuthClient`.
- 4 `PUT /{id}` → `actualizar(...)`: `nombre/email/rol/habilitado/nuevaPassword`.
- 5 `POST /{id}/toggle` → `toggle(id)`: `usuarioService.alternarHabilitado(id)` (deshabilita/rehabilita sin borrar).
- 6 `DELETE /{id}` → `eliminar(id, @AuthenticationPrincipal AuthPrincipal principal)`: si `principal.id() == id` → `IllegalArgumentException("No puedes eliminar tu propio usuario")`; si no → `usuarioService.eliminarUsuario(id)`. **Regla de negocio de seguridad:** un admin no se autodestruye.

```
@RestController @RequestMapping("/api/admin") @PreAuthorize("hasRole('ADMIN')") ·  
AdminStatsController
```

- 1 GET /stats → stats() : cuenta usuarios por rol localmente y pide juegos/secciones a catalogoClient.stats() (si está caído vienen en -1). Devuelve StatsDTO.

InternalUsuariosController

usuarios-service/.../web/InternalUsuariosController.java

@RestController @RequestMapping("/internal/usuarios") . La consumen auth-service (login, refresh, Google) y catalogo-service (resolver vendedores). Detrás de InternalTokenFilter .

```
GET /by-username/{username} → UsuarioDTO porUsername(...)
```

- 1 UsuarioDTO.from(usuarioService.buscarPorUsername(username)) . Es la fuente de rol + habilitado del login.

```
GET /{id} → porId · GET ?ids=1,2,3 → porIds · GET ?usernames=a,b → porUsernames
```

- 1 Resolución individual y en lote. porUsernames la usa el catálogo para pintar el vendedor de cada juego en **una** llamada.

```
POST /google → UsuarioDTO buscarOCrearGoogle(@RequestBody Map body)
```

- 1 usuarioService.buscarOCrearDesdeGoogle(email, nombre) — crea el perfil con rol COMPRADOR si es la primera vez.

AuthClient (usuarios-service → auth-service)

usuarios-service/.../client/AuthClient.java

@Component . RestClient con X-Internal-Token por defecto. Sin circuit breaker con fallback silencioso a propósito: si auth-service no responde en un alta, la operación debe **fallar** (y usuarios-service compensa borrando el perfil), no dejar un usuario a medio crear.

```
@Retry void crearCredencial(String username, String email, String password)
```

- 1 POST /internal/credenciales con {username, email, password}.

```
@Retry void cambiarPassword(String username, String nuevaPassword) · @Retry void  
eliminarCredencial(String username)
```

- 1 PUT /internal/credenciales/{u}/password y DELETE /internal/credenciales/{u} respectivamente.

Reglas por rol para publicar/comprar + autorización "a nivel de dato" (¿es tu juego?), que no se puede expresar por URL.

SecurityConfig

catalogo-service/.../config/SecurityConfig.java

```
@Configuration @EnableWebSecurity @EnableMethodSecurity(prePostEnabled = true).
```

```
@Bean SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationFilter
jwtAuthenticationFilter)
```

- 1 csrf/formLogin/httpBasic disable ; sessionManagement(STATELESS) .
- 2 **Reglas** authorizeHttpRequests , en orden (el orden manda: la primera que matchea gana):
 - /actuator/health* → permitAll
 - /uploads/** → permitAll (imágenes de portada, públicas)
 - /internal/** → hasRole("INTERNAL")
 - POST /api/juegos/*/comprar → hasRole("COMPRADOR")
 - GET /api/compras → hasRole("COMPRADOR")
 - POST|PUT|DELETE /api/juegos/** → hasAnyRole("VENDEDOR", "ADMIN")
 - POST /api/secciones , DELETE /api/secciones/** → hasRole("ADMIN")
 - /api/** → authenticated (p. ej. GET /api/juegos : cualquier usuario logueado ve el catálogo)
 - anyRequest() → authenticated
- 3 exceptionHandling : 401 / 403 JSON.
- 4 addFilterBefore(InternalTokenFilter) + addFilterBefore(jwtAuthenticationFilter) .

JuegoController — rol + pertenencia

catalogo-service/.../web/JuegoController.java

```
GET / → listar(@AuthenticationPrincipal AuthPrincipal) · GET /{id} → obtener(...)
```

- 1 Sólo authenticated . mapper.aDtos(juegoService.listarTodos(), principal) — el principal se usa para calcular puedeEditar / comprado de cada juego (server-side).

```
@PreAuthorize("hasAnyRole('VENDEDOR','ADMIN')") POST / → crear(...multipart...,
@AuthenticationPrincipal AuthPrincipal principal)
```

- 1 Resuelve la sección si vino `seccionId`.
- 2 `juegoService.crear(nombre, descripcion, precio, stock, seccion, principal.username(), imagen)` — el vendedor dueño se toma del **token**, no de un campo del form.
- 3 `return mapper.aDto(juego, principal)`.

```
@PreAuthorize("hasAnyRole('VENDEDOR','ADMIN')") PUT /{id} → actualizar(...) · DELETE /{id} →
eliminar(...) · POST /{id}/stock → agregarStock(...)
```

- 1 `existente = juegoService.buscarPorId(id)`.
- 2 `verificarPropietarioOAdmin(existente, principal)` — **acá** se hace el control a nivel de dato, además del rol.
- 3 Recién entonces llama al servicio (`actualizar` / `eliminar` / `agregarStock`).

```
private void verificarPropietarioOAdmin(Juego juego, AuthPrincipal principal)
```

- 1 Si `!juegoService.puedeGestionar(juego, principal)` → `throw new AccessDeniedException("No podes modificar juegos de otro vendedor")` → el handler lo vuelve `403`.

JuegoService.puedeGestionar() — la regla de pertenencia

catalogo-service/.../service/JuegoService.java

```
boolean puedeGestionar(Juego juego, AuthPrincipal usuarioActual)
```

- 1 `return usuarioActual.isAdmin() || juego.getVendedorUsername().equals(usuarioActual.username())`.
- 2 El ADMIN modera cualquier juego; el VENDEDOR sólo los suyos. Se compara contra el `username` del JWT — ya no hay entidad `Usuario` que consultar.

CompraController

catalogo-service/.../web/CompraController.java

```
@PreAuthorize("hasRole('COMPRADOR')") POST /api/juegos/{id}/comprar → comprar(@PathVariable
Long id, @AuthenticationPrincipal AuthPrincipal principal)
```

- 1 `CompraDTO.from(compraService.comprar(id, principal))` → `201 Created`. El comprador se toma del token.

```
@PreAuthorize("hasRole('COMPRADOR')") GET /api/compras → misCompras(@AuthenticationPrincipal  
AuthPrincipal principal)
```

- 1 `compraService.misCompras(principal.username())` — se filtra por el `username` del JWT, **no** por un parámetro que el cliente pueda cambiar. Cada quien ve sólo lo suyo.

JuegoDtoMapper — **puedeEditar** lo decide el servidor

catalogo-service/.../web/JuegoDtoMapper.java

```
List<JuegoDTO> aDtos(List<Juego> juegos, AuthPrincipal principal)
```

- 1 Junta los `vendedorUsername` distintos y los resuelve en **una** llamada:
`usuariosClient.porUsernames(usernames)`.
- 2 `comprados = comprados(principal)` (ver abajo).
- 3 Mapea cada juego con `aDto(j, vendedores, comprados, principal)`.

```
private Set<Long> comprados(AuthPrincipal principal)
```

- 1 Si `principal == null` o `rol != "COMPRADOR"` → `Set.of()` (la marca "comprado" sólo aplica a compradores).
- 2 Si no → `compraService.juegosComprados(principal.username())`.

```
private JuegoDTO aDto(Juego j, Map vendedores, Set<Long> comprados, AuthPrincipal principal)
```

- 1 Resuelve el `VendedorDTO` (o uno "degradado" si usuarios-service no lo devolvió).
- 2 Setea `puedeEditar = juegoService.puedeGestionar(j, principal)` y `comprado = comprados.contains(j.getId())`. **El frontend nunca decide esto**: sólo pinta el botón según el flag que le llega.

8 Claves RSA (`keys/`)

Archivo	Formato	Quién lo usa	Para qué
<code>jwt-private.pem</code>	PKCS#8	sólo auth-service	firmar los access tokens (RS256)
<code>jwt-public.pem</code>	X.509 SPKI	gateway, usuarios-service, catalogo-service	validar la firma

Se copian a `src/main/resources/` de cada servicio que las necesita (por eso hay varias copias del `.pem` en el repo). `generate-keys.sh` las regenera con `openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 + openssl pkey -pubout`.

LAS CLAVES DEL REPO SON DE DEMO

Están commiteadas a propósito para que `docker compose up` funcione sin pasos previos (mismo criterio que el `demo-secret` del monolito). En producción: generar el par fuera del repo, dejar `jwt-private.pem` sólo en auth-service (secret montado / variable de entorno) y distribuir `jwt-public.pem` como recurso de sólo lectura.

9 Tests que fijan el comportamiento de seguridad

AuthFlowIntegrationTest (auth-service)

auth-service/src/test/java/com/gamestore/auth/AuthFlowIntegrationTest.java

@SpringBootTest @AutoConfigureMockMvc. Mockea sólo UsuariosClient; el resto corre de verdad (H2, JWT firmado con la clave del classpath, DataInitializer, LoginAttemptListener por eventos).

- login_ok_devuelveAccessYRefreshToken: POST /api/auth/login con admin/admin123 → 200 con token, refreshToken y usuario.rol == "ADMIN".
- login_passwordIncorrecta_401: mensaje exacto "Usuario o contraseña incorrectos".
- login_cuentaDeshabilitada_401: con usuariosClient devolviendo habilitado=false → 401 "Esta cuenta esta deshabilitada".
- login_5intentosFallidos_bloqueanLaCuenta: 5 intentos malos y el 6º con la password correcta también da 401 (mensaje contiene "bloqueada"); verifica bloqueadoHasta en el futuro.
- refresh_rota_yElRefreshViejoQuedaInvalido: tras /refresh, el token viejo ya no sirve (401).
- dosFactores_setupEnableYLuegoElLoginExigeElCodigo: setup → enable con TOTP válido → el login sin código responde 401 {requiere2fa:true}; con código válido, 200; luego disable.
- dosFactores_enableConCodigoInvalido_400: enable con "000000" → 400.

CatalogoAuthorizationTest (catalogo-service)

catalogo-service/src/test/java/com/gamestore/catalogo/CatalogoAuthorizationTest.java

@SpringBootTest @AutoConfigureMockMvc @Transactional. Como este servicio no tiene la clave privada, el JWT se **simula** inyectando un AuthPrincipal directo en el contexto (SecurityMockMvcRequestPostProcessors.authentication(...)).

- vendedor_puedeBorrarSuPropioJuego: DELETE /api/juegos/{id} como vendedor1 sobre un juego suyo → 204.
- vendedor_noPuedeBorrarJuegoDeOtroVendedor: mismo verbo sobre un juego de vendedor2 → 403 (lo corta verificarPropietarioOAdmin).
- admin_puedeBorrarCualquierJuego: como admin sobre juego ajeno → 204.
- comprador_noPuedePublicar: POST /api/juegos como COMPRADOR → 403 (lo corta el @PreAuthorize / la regla de URL).

10 Tablas de referencia y notas de producción

Cadena de filtros por servicio (orden efectivo)

#	api-gateway (WebFlux)	auth / usuarios / catalogo (servlet)
1	<code>GatewayAuthFilter</code> (order -1): limpia headers, valida JWT, agrega <code>X-Auth-*</code>	<code>InternalTokenFilter</code> (sólo <code>/internal/**</code>): valida <code>X-Internal-Token</code> → <code>ROLE_INTERNAL</code>
2	Filtros de ruteo de Spring Cloud Gateway	<code>JwtAuthenticationFilter</code> : <code>Bearer</code> → <code>AuthPrincipal</code> en el <code>SecurityContext</code>
3	—	<code>authorizeHttpRequests</code> : reglas por path/verbo/rol
4	—	<code>@PreAuthorize</code> en el controller (rol) + chequeo de pertenencia en el servicio

Matriz endpoint → rol requerido

Endpoint	Servicio	Requisito
<code>POST /api/auth/login refresh logout</code> , <code>GET /api/auth/me</code>	auth	público
<code>/api/auth/2fa/**</code>	auth	autenticado (cualquier rol)
<code>/oauth2/**</code> , <code>/login/oauth2/**</code>	auth	público (flujo Google)
<code>/api/usuarios/**</code> , <code>/api/admin/**</code>	usuarios	<code>ROLE_ADMIN</code>
<code>GET /api/juegos</code> , <code>GET /api/juegos/{id}</code> , <code>GET /api/secciones</code>	catalogo	autenticado
<code>POST/PUT/DELETE /api/juegos/**</code>	catalogo	<code>ROLE_VENDEDOR</code> o <code>ROLE_ADMIN</code> + ser dueño (o admin)
<code>POST /api/juegos/{id}/comprar</code> , <code>GET /api/compras</code>	catalogo	<code>ROLE_COMPRADOR</code>
<code>POST /api/secciones</code> , <code>DELETE /api/secciones/**</code>	catalogo	<code>ROLE_ADMIN</code>
<code>GET /uploads/**</code>	catalogo	público
<code>/internal/**</code> (todos)	todos	header <code>X-Internal-Token</code> → <code>ROLE_INTERNAL</code>
<code>/actuator/health</code>	todos	público

Claims del access token

Claim	Valor	Uso
iss	gamestore-auth	obligatorio en la validación (requireIssuer)
sub	username	AuthPrincipal.username
uid	id numérico	AuthPrincipal.id (comparaciones de pertenencia)
rol	ADMIN / VENDEDOR / COMPRADOR	→ ROLE_* en authorities()
iat / exp	ahora / +15 min	vida corta; la sesión larga la sostiene el refresh token

Puntos a endurecer antes de producción

CHECKLIST

- Regenerar el par RSA fuera del repo; `jwt-private.pem` sólo en auth-service (secret montado).
- `app.internal.secret` : valor real por entorno, distinto del `dev-internal-secret-cambiar-en-prod` . Idealmente además mTLS / network policies entre servicios.
- `GATEWAY_ALLOWED_ORIGINS` : fijar el origen exacto del frontend (no `*`).
- Google OAuth2: `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` reales; revisar redirect URIs.
- Rate-limiting en el gateway para `/api/auth/login` (el bloqueo por intentos es por-cuenta, no por-IP).
- Persistir/propagar `totpSecret` cifrado en reposo (hoy va en claro en la columna).
- Rotar y expirar refresh tokens también del lado servidor por inactividad; limpiar los revocados/expirados con un job.